

Chapter 13: Data Storage Structures

Outline

File Organization

- Fixed-Length Records

- Variable-Length Records

Organization of Records in Files

- Heap File Organization

- Sequential File Organization

- Multitable Clustering File Organization

- Table Partition

Data-Dictionary Storage

Storage Access & Buffer manager

- LRU strategy

- Clock algorithm

Columnar Representation

File Organization

The **database** is stored as a collection of **files**.

Each file is a sequence of **records**.

A record is a sequence of **fields**.

One approach:

- assume **record size is fixed**

- each file has records of one particular type only**

- different files are used for different relations**

This case is easiest to implement; will consider variable length records later.

We assume that records are smaller than a disk block

.

Fixed-Length Records

Simple approach:

Store **record i** starting from byte $n * (i - 1)$, where n is the size of each record.

Record access is simple but records may cross blocks

- Modification: do not allow records to cross block boundaries

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

Deletion of record i : alternatives:

move records $i + 1, \dots, n$ to $i, \dots, n - 1$

move record n to i

do not move records, but link all free records on a *free list*

Record 3 deleted

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000



Fixed-Length Records

Deletion of record i : alternatives:

move records $i + 1, \dots, n$ to $i, \dots, n - 1$

move record n to i

do not move records, but link all free records on a *free list*

Record 3 deleted and replaced by record 11

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000



Fixed-Length Records

Deletion of record i : alternatives:

move records $i + 1, \dots, n$ to $i, \dots, n - 1$

move record n to i

do not move records, but link all free records on a *free list*

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Variable-Length Records

Variable-length records arise in database systems in several ways:

- Storage of **multiple record types** in a file.

- Record types that allow **variable lengths for one or more fields** such as strings (**varchar**)

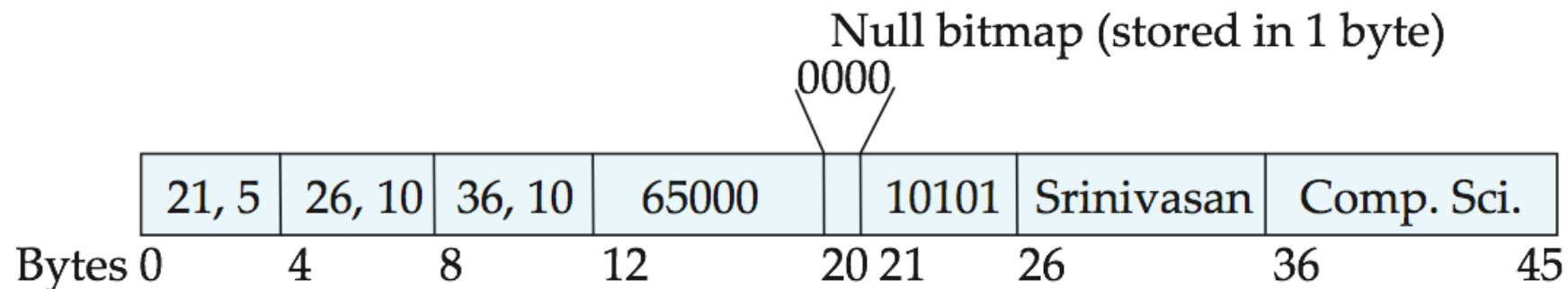
- Record types that allow **repeating fields** (used in some older data models).

Attributes are stored in order

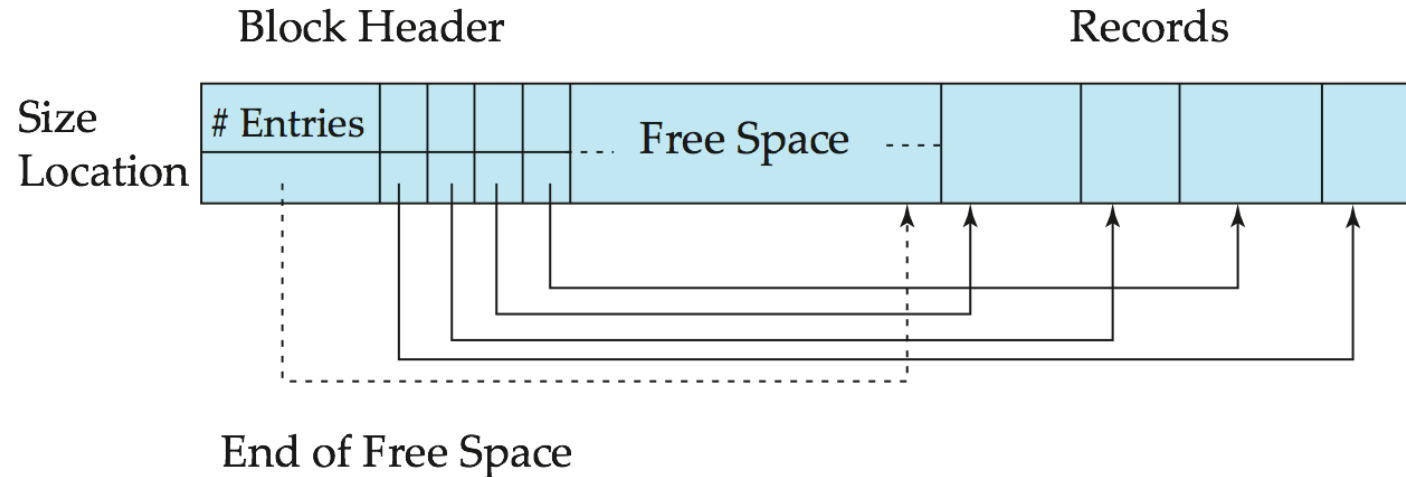
Variable length attributes represented by **fixed size (offset, length)**, with actual data stored after all fixed length attributes

Null values represented by **null-value bitmap (空位图)**

Instructor(id, name, dept_name, salary)



Variable-Length Records: Slotted Page Structure



Slotted page (分槽页) header contains:

- number of record entries

- end of free space in the block

- location and size of each record

Records can be moved around within a page to keep them contiguous with no empty space between them; entry in the header must be updated.

Record pointers should not point directly to record — instead they should point to the entry for the record in header.

Organization of Records in Files

Heap – record can be placed anywhere in the file where there is space

Sequential – store records in sequential order, based on the value of the **search key** of each record

In a **multitable clustering file organization** records of several different relations can be stored in the same file

Motivation: store related records on the same block to minimize I/O

B+-tree file organization

Ordered storage even with inserts/deletes

Hashing – a hash function computed on search key; the result specifies in which block of the file the record should be placed

Heap File Organization

Records can be placed anywhere in the file where there is free space

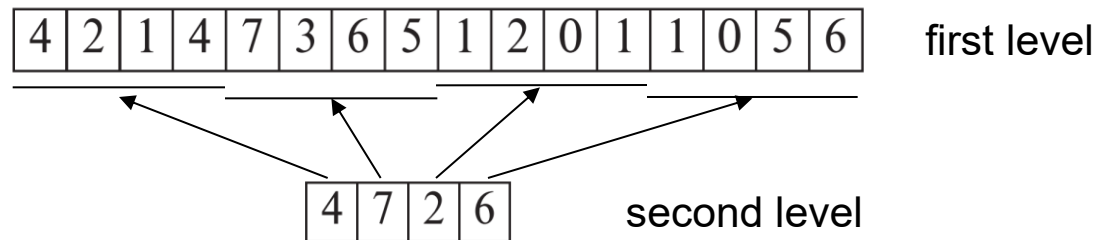
Records usually do not move once allocated

Important to be able to efficiently find free space within file

Free-space map

Array with 1 entry per block. Each entry is a few bits to a byte, and records fraction of block that is free

In example below, **3 bits per block**, value divided by 8 indicates



Can have **second-level free-space map**

In example above, each entry stores maximum from 4 entries of first-level free-space map

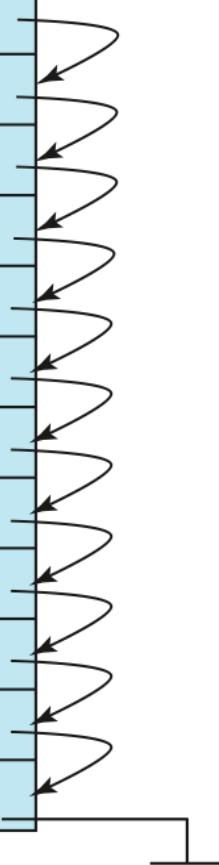
Free space map written to disk periodically, OK to have wrong (old) values for some entries (will be detected and fixed)

Sequential File Organization

Suitable for applications that require **sequential processing** of the entire file

The records in the file are ordered by a **search-key**

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

A diagram illustrating sequential file organization. It shows a table with 12 rows of records. To the right of the table, a vertical line with arrows pointing left indicates sequential access from top to bottom. At the bottom right, there is a ground symbol consisting of two horizontal lines of decreasing width.

Sequential File Organization (Cont.)

Deletion – use pointer chains

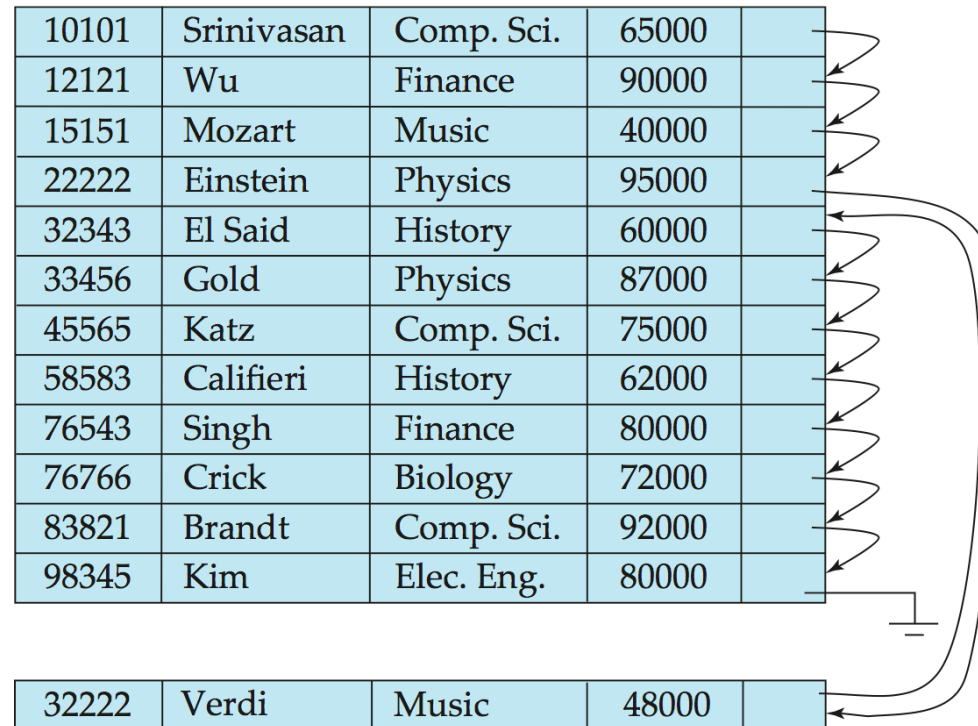
Insertion –locate the position where the record is to be inserted

if there is free space insert there

if no free space, insert the record in an **overflow block**

In either case, pointer chain must be updated

Need to **reorganize** the file
from time to time to restore
sequential order



Multitable Clustering File Organization

Store several relations in one file using a **multitable clustering** file organization

department

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

multitable clustering
of *department* and
instructor

Comp. Sci.	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000
Physics	Watson	70000
33456	Gold	87000

Multitable Clustering File Organization (cont.)

good for queries involving **join** *department* ⋈ *instructor*, and for queries involving one single department and its instructors

bad for queries involving only *department*

results in variable size records

Can add pointer chains to link records of a particular relation

Comp. Sci.	Taylor	100000	
45564	Katz	75000	
10101	Srinivasan	65000	
83821	Brandt	92000	
Physics	Watson	70000	
33456	Gold	87000	



Table Partitioning

Table partitioning: Records in a relation can be partitioned into smaller relations that are stored separately

E.g., *transaction* relation may be partitioned into *transaction_2018*, *transaction_2019*, etc.

Queries written on *transaction* must access records in all partitions

Unless query **has** a selection such as *year*=2019, in which case only one partition is needed

Partitioning

Reduces costs of some operations such as free space management

Allows different partitions to be stored on different storage devices

- ▶ E.g., *transaction* partition for current year on SSD, for older years on magnetic disk

Data Dictionary Storage

The **Data dictionary** (also called **system catalog**) stores **metadata**; that is, data about data, such as

Information about **relations**

- names of relations

- names, types and lengths of attributes of each relation

- names and definitions of views

- integrity constraints

User and accounting information, including passwords

Statistical and descriptive data

- number of tuples in each relation

Physical file organization information

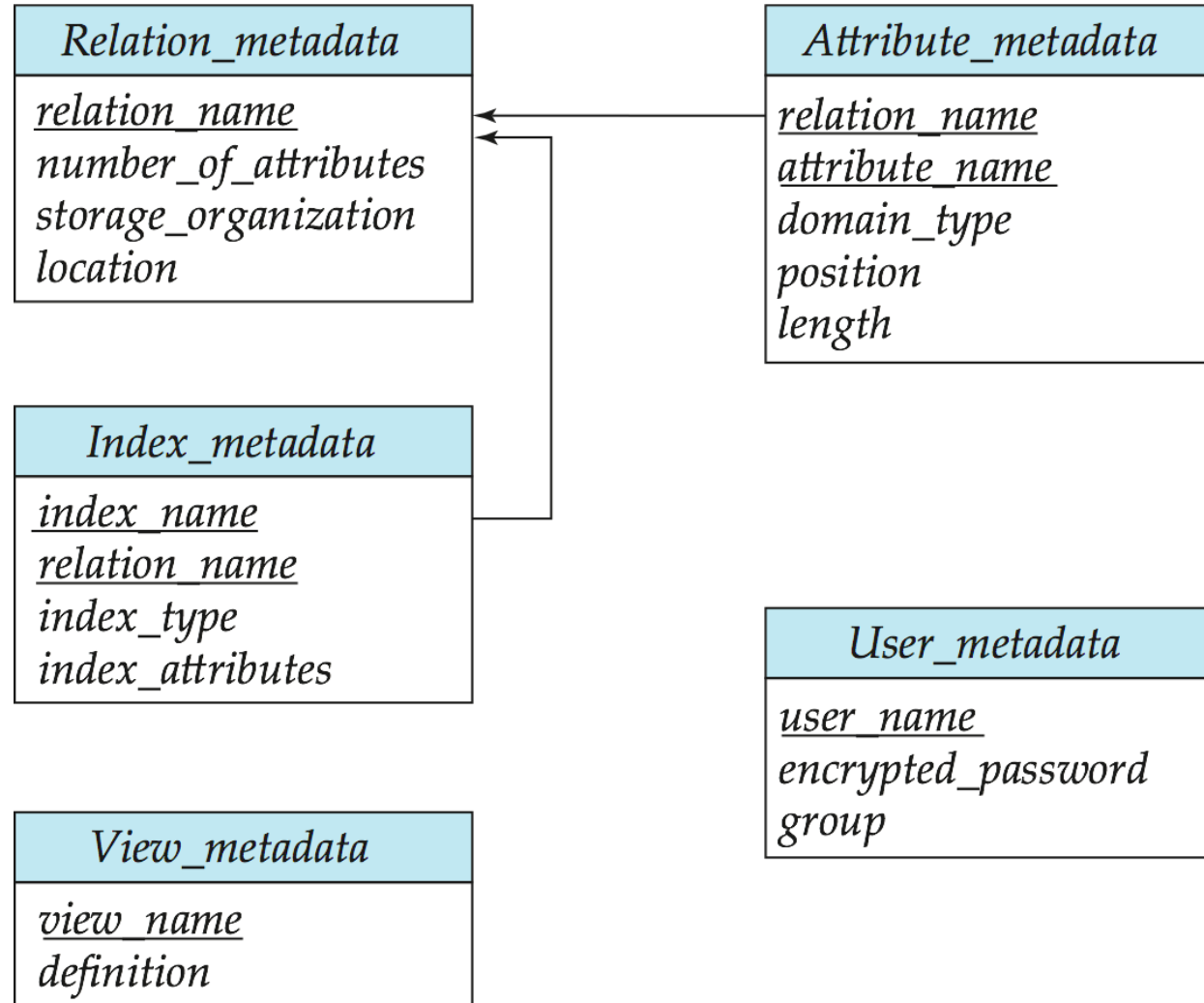
- How relation is stored (sequential/hash/...)

- Physical location of relation

Information about **indices**

Relational Representation of System Metadata

Relational representation on disk
Specialized data structures designed for efficient access, in memory



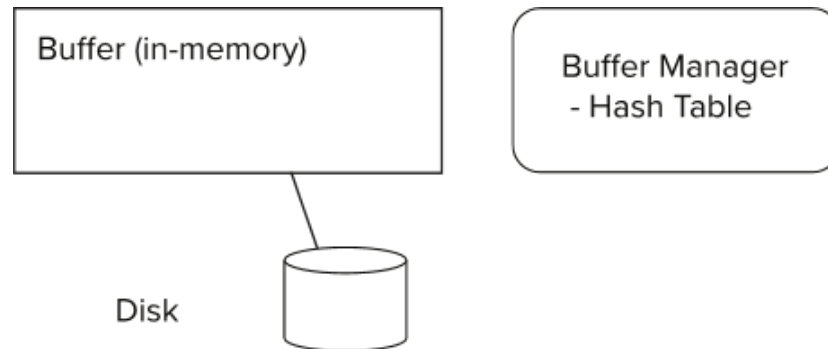
Storage Access

Blocks are units of both storage allocation and data transfer.

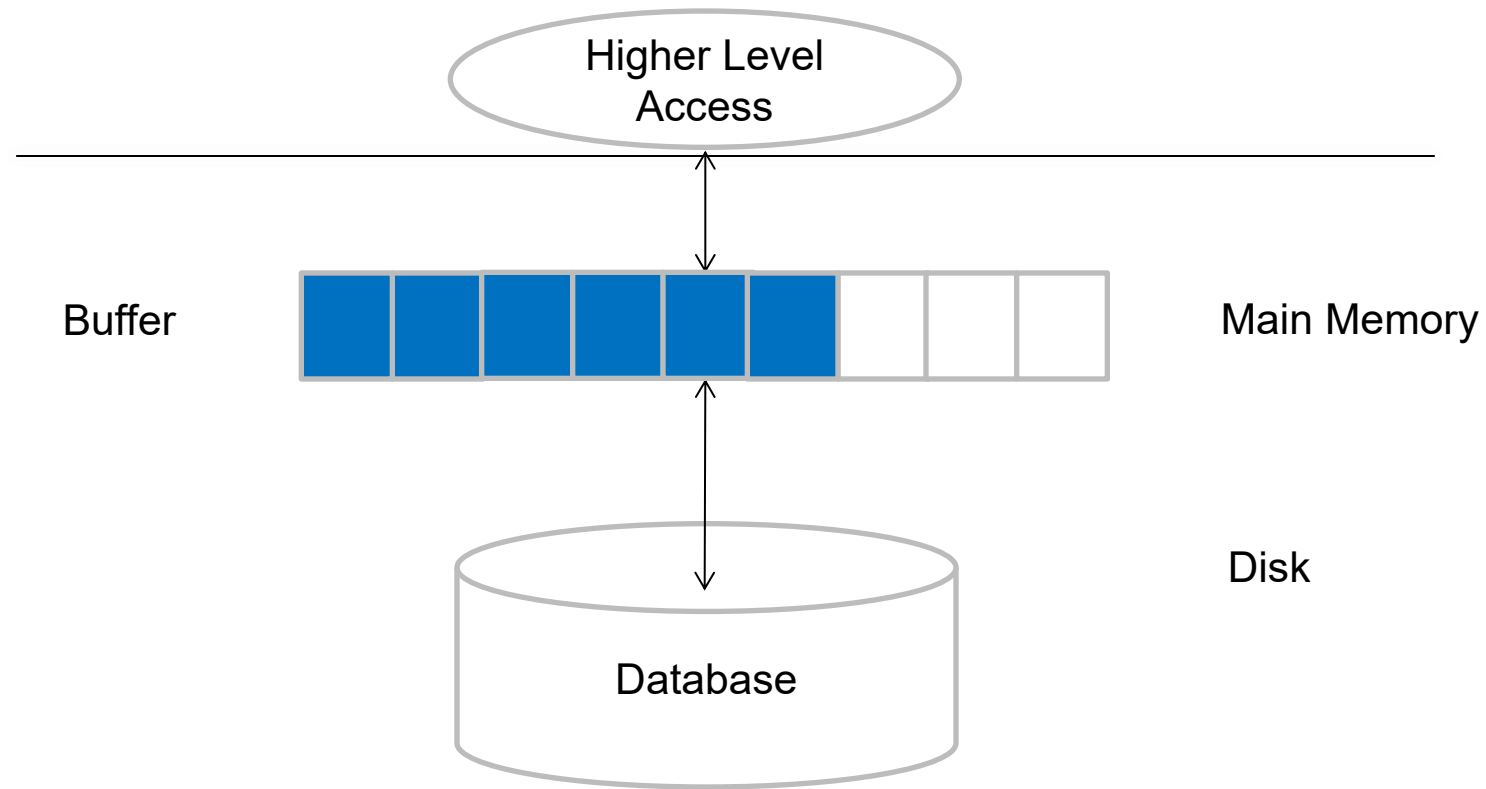
Database system seeks to minimize the number of block transfers between the disk and memory. We can reduce the number of disk accesses by keeping as many blocks as possible in main memory.

Buffer – portion of main memory available to store copies of disk blocks.

Buffer manager – subsystem responsible for allocating buffer space in main memory.

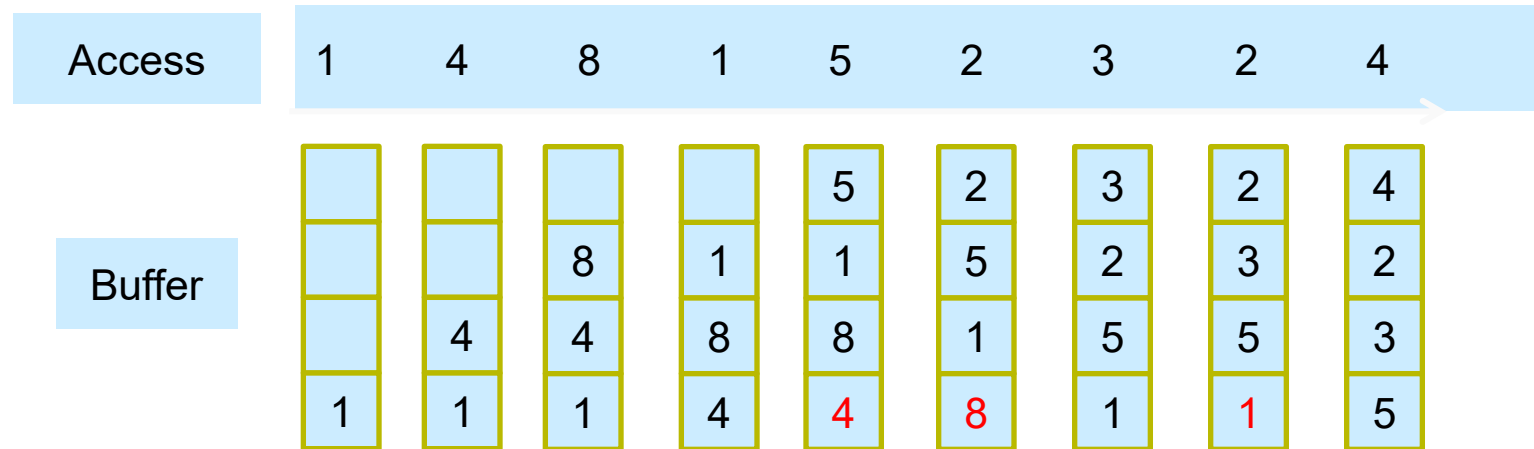


Buffer Manager in DBMS



LRU Example

- Supposing there are 4 buffer pages.
- Initial is empty occupied.
- The access sequence is 1, 4, 8, 1, 5, 2, 3, 2, 4.
- According to the LRU policy , there are 3 times replacement occurred.



Buffer Manager

Programs call on the buffer manager when they need a block from disk.

1. If the block is already in the buffer, buffer manager returns the address of the block in main memory
2. If the block is not in the buffer, the buffer manager
 - ① Allocates space in the buffer for the block
 - a) Replacing (throwing out) some other block, if required, to make space for the new block.
 - b) Replaced block written back to disk only if it was modified since the most recent time that it was written to/fetched from the disk.
 - ② Reads the block from the disk to the buffer, and returns the address of the block in main memory to requester.

Buffer Manager

Buffer replacement strategy (details coming up!)

Pinned block: memory block that is not allowed to be written back to disk

Pin done before reading/writing data from a block

Unpin done when read /write is complete

Multiple concurrent pin/unpin operations possible

- ▶ Keep a pin count, buffer block can be evicted only if **pin count = 0**

Shared and exclusive locks on buffer

Needed to prevent concurrent operations from reading page contents as they are moved/reorganized, and to ensure only one move/reorganize at a time

Readers get **shared lock**, updates to a block require **exclusive lock**

Locking rules:

- ▶ Only one process can get exclusive lock at a time
- ▶ Shared lock cannot be concurrently with exclusive lock
- ▶ Multiple processes may be given shared lock concurrently

Buffer-Replacement Policies

LRU strategy - replace the block **least recently used** .

Idea behind LRU – use past pattern of block references as a predictor of future references

Queries have well-defined **access patterns** (such as sequential scans), and a database system can use the information in a user's query to **predict future references**

LRU can be a bad strategy for certain access patterns involving **repeated scans** of data

For example: when computing the **join** of 2 relations r and s by a nested loops

```
for each tuple  $tr$  of  $r$  do
  for each tuple  $ts$  of  $s$  do
    if the tuples  $tr$  and  $ts$  match ...
```

Mixed strategy with hints on replacement strategy provided by the query optimizer is preferable

Buffer-Replacement Policies (Cont.)

Toss-immediate strategy – frees the space occupied by a block as soon as the final tuple of that block has been processed

Most recently used (MRU) strategy – system must pin the block currently being processed. After the final tuple of that block has been processed, the block is unpinned, and it becomes the most recently used block.

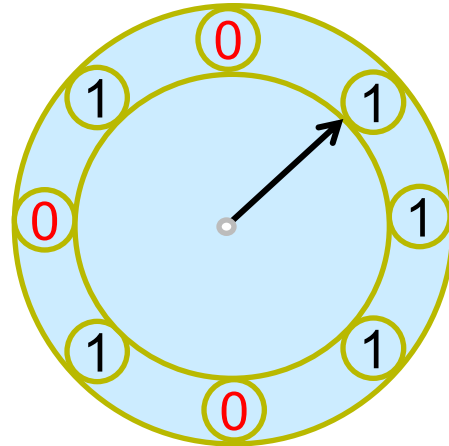
Buffer managers also support **forced output** of blocks for the purpose of recovery

Clock: An approximation of LRU

Arrange block into a **cycle**, store one **reference_bit** per block

When **pin_count** reduces to 0, set **reference_bit = 1**

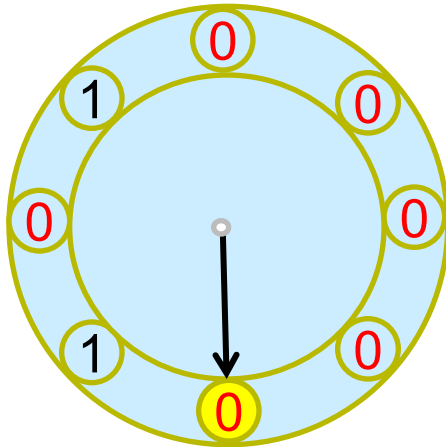
reference_bit as the 2nd chance bit



Clock: An approximation of LRU

When replacement necessary

```
do for each block in cycle {  
    if (reference_bit == 1)  
        set reference_bit=0;  
    else if (reference_bit == 0)  
        choose this block for replacement;  
} until a page is chosen;
```



Column-Oriented Storage

Also known as **columnar representation**

Store each attribute of a relation separately

Example

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Columnar Representation

Benefits:

- Reduced IO if only some attributes are accessed

- Improved CPU cache performance

- Improved compression

- Vector processing** on modern CPU architectures

Drawbacks

- Cost of tuple reconstruction from columnar representation

- Cost of tuple deletion and update

- Cost of decompression

Columnar representation found to be more efficient for **decision support** than row-oriented representation

Traditional **row-oriented representation** preferable for transaction processing

Some databases support both representations

- Called **hybrid row/column stores**

Columnar File Representation

ORC(Optimized Row Columnar) and Parquet

File formats of Hadoop HDFS

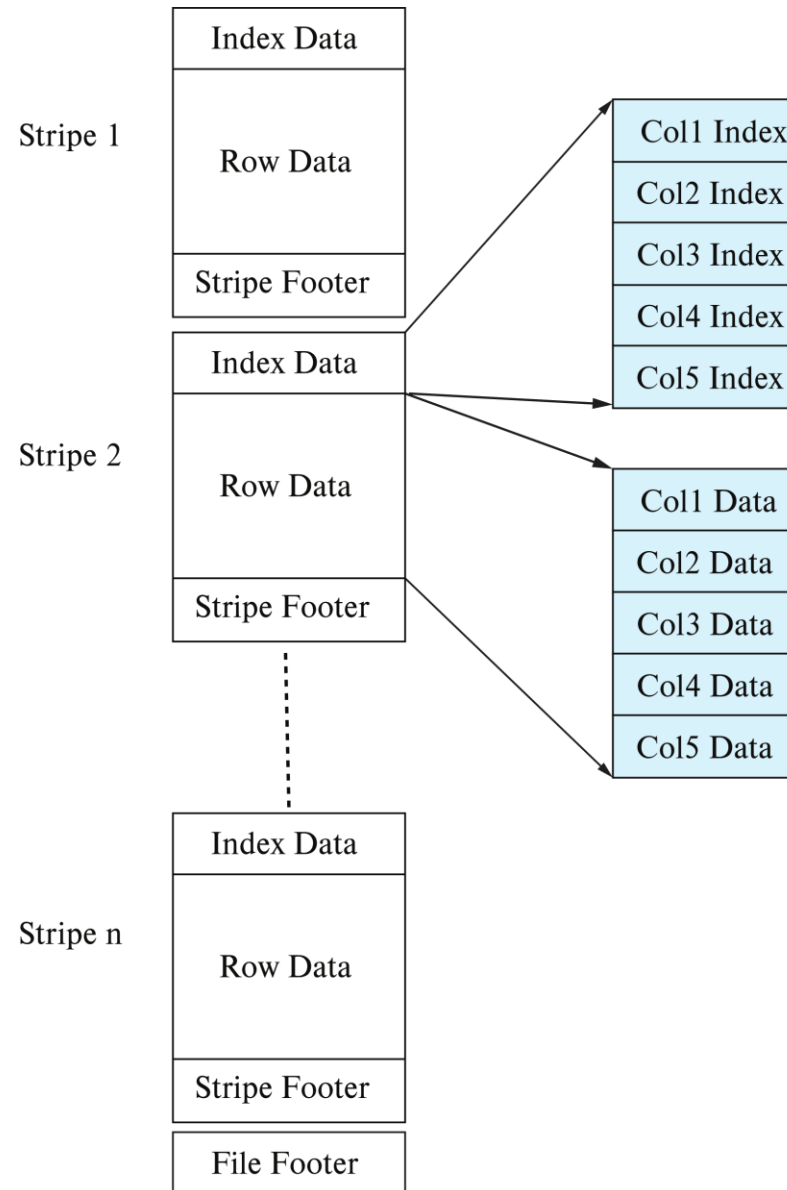
Apache ORC is an open source, a self-describing type-aware columnar file format designed for Hadoop workloads.

Apache Parquet is an open source, column-oriented data file format designed for efficient data storage and retrieval.

File formats with columnar storage inside file

Very popular for big-data applications

Orc file format shown on right:



End of Chapter 13